

ANSH THAPA

B.TECH CSE

CU24250224

Experiment No. 7

Motion Estimation using Optical Flow Algorithms in Video Sequences

Aim

To implement optical flow algorithms for estimating the motion of objects between consecutive video frames and analyze motion patterns for dynamic scene understanding using Python and OpenCV.

Purpose

Optical Flow is a fundamental computer vision technique used to estimate the apparent motion of objects, surfaces, or camera movement by analyzing consecutive frames in a video sequence. Motion estimation plays a vital role in applications such as autonomous driving, video surveillance, robotics, gesture recognition, sports analytics, and augmented reality.

In this experiment, students will implement Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow algorithms to estimate object motion, visualize motion vectors, and compare the performance of both techniques on real-world video sequences.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Understanding of video processing and image gradients
- Jupyter Notebook, Google Colab, or Visual Studio Code

```
In [20]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load video
cap = cv2.VideoCapture("/content/20090-307115630_medium.mp4")

# Check if video is opened
if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video loaded successfully.")
```

Video loaded successfully.

```
In [21]: ret, old_frame = cap.read()

if not ret:
    print("Could not read the first frame.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

ret, new_frame = cap.read()

if ret:
    new_gray = cv2.cvtColor(new_frame, cv2.COLOR_BGR2GRAY)

    plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.imshow(old_gray, cmap='gray')
    plt.title("Previous Frame")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(new_gray, cmap='gray')
    plt.title("Current Frame")
    plt.axis("off")

    plt.show()
```

Previous Frame



Current Frame



```
In [22]: # Parameters for Shi-Tomasi corner detection
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Detect good feature points
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

# Calculate optical flow
p1, st, err = cv2.calcOpticalFlowPyrLK(
    old_gray,
    new_gray,
    p0,
    None,
    **lk_params
)

# Select good points
good_new = p1[st == 1]
good_old = p0[st == 1]

print("Number of tracked points:", len(good_new))
```

Number of tracked points: 100

```
In [23]: # Create a copy of the current frame
lk_output = new_frame.copy()

# Draw motion vectors
for new, old in zip(good_new, good_old):
    x1, y1 = new.ravel()
    x2, y2 = old.ravel()

    cv2.arrowedLine(
        lk_output,
        (int(x2), int(y2)),
        (int(x1), int(y1)),
        (0, 255, 0),
        2
    )

    cv2.circle(
        lk_output,
        (int(x1), int(y1)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Optical Flow")
plt.axis("off")
plt.show()
```

Lucas-Kanade Optical Flow



```
In [24]: # Calculate dense optical flow
flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    new_gray,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

print("Dense optical flow calculated successfully.")
```

Dense optical flow calculated successfully.

```
In [25]: # Convert flow into magnitude and angle
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# Create HSV image
hsv = np.zeros_like(old_frame)
hsv[..., 1] = 255

# Direction represented by hue
hsv[..., 0] = angle * 180 / np.pi / 2

# Magnitude represented by brightness
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert HSV to BGR
dense_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(dense_flow, cv2.COLOR_BGR2RGB))
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```

Farneback Dense Optical Flow



Step 7: Compare Lucas-Kanade and Farneback

Feature	Lucas-Kanade	Farneback
Type	Sparse	Dense
Tracking	Selected feature points	Almost every pixel
Speed	Faster	Slower
Computational Cost	Low	High
Accuracy	Good for selected points	Good for overall motion
Robustness	Good for textured features	Good for general motion
Output	Motion vectors of points	Motion field
Best Use	Object tracking	Motion analysis

Step 8: Effect of Speed, Lighting and Camera Motion

1. Object Speed

- Slow-moving objects are easier to track.
- Very fast objects can produce large displacement.
- Large displacement may reduce Lucas-Kanade tracking accuracy.
- Farneback can also become less reliable when motion is very large.

2. Lighting Conditions

- Optical flow depends on changes in image intensity.
- Sudden changes in brightness can cause incorrect motion estimation.
- Stable lighting generally produces better results.

3. Camera Motion

- If the camera moves, many pixels appear to move.
- This can make it difficult to distinguish camera motion from object motion.
- Background motion can therefore affect both algorithms.

Step 9: Real-World Applications Lucas-Kanade

Suitable for:

- Object tracking
- Feature tracking
- Motion estimation
- Video stabilization
- Augmented reality

- Farneback

Suitable for:

- Surveillance
- Crowd-motion analysis
- Traffic monitoring
- Autonomous navigation
- Dynamic scene analysis

Step 10: Observations and Conclusion

Observations

1. Lucas-Kanade successfully tracks selected feature points between consecutive frames.
2. Farneback calculates dense motion information across the image.
3. Lucas-Kanade is generally faster and requires less computation.
4. Farneback provides more detailed motion information but is computationally more expensive.
5. High object speed, changing illumination, and camera movement can reduce optical-flow accuracy.
6. The choice of algorithm depends on the application and required level of motion information.

OPTICAL FLOW – QUESTIONS AND ANSWERS

1. What is Optical Flow? How is it used in computer vision?

Answer: Optical Flow is the apparent movement of pixels or objects between two consecutive video frames. It is used to find the direction and speed of motion. It is useful for object tracking, video analysis, surveillance, and autonomous vehicles.

2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Answer: Lucas-Kanade is a Sparse Optical Flow algorithm. It first detects important feature points such as corners. Then, it compares these points in consecutive frames and calculates their movement. It assumes that nearby pixels have similar motion.

3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Answer: Dense Optical Flow calculates the motion of almost every pixel in an image.

Sparse Optical Flow tracks only selected feature points, while Dense Optical Flow estimates motion for almost all pixels. Sparse Optical Flow is faster, whereas Dense Optical Flow provides more detailed motion information.

4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Answer: Lucas-Kanade is a Sparse Optical Flow algorithm that tracks selected feature points. It is faster and requires less computational power. Farneback is a Dense Optical Flow algorithm that estimates motion for almost every pixel. It provides more detailed motion information but requires more computation.

5. What assumptions are made while computing optical flow?

Answer: The main assumptions are:

1. The brightness of an object remains approximately constant.
2. Objects move only a small distance between consecutive frames.
3. Nearby pixels have similar motion.
4. The image contains enough features or texture for tracking.
5. What factors can affect the accuracy of optical flow estimation?

Answer: The accuracy of optical flow can be affected by fast object movement, sudden lighting changes, camera movement, motion blur, image noise, low video quality, and object occlusion.

7. Mention five real-world applications of optical flow in computer vision.

Answer: Five real-world applications of optical flow are:

1. Object tracking

2. Video surveillance
3. Autonomous driving
4. Video stabilization
5. Action recognition
6. Why are grayscale images generally used for optical flow computation?

Answer: Grayscale images contain only brightness or intensity information, which makes calculations simpler and faster. Optical flow mainly depends on changes in pixel intensity, so color information is usually not required.

9. What are the limitations of optical flow algorithms in real-world environments?

Answer: Optical flow algorithms can have difficulty with very fast motion, changing lighting conditions, camera movement, motion blur, object occlusion, and low-quality images. Dense optical flow can also require more computational power.

10. How does optical flow contribute to applications such as autonomous driving, video surveillance, and action recognition?

Answer: Optical flow helps computer vision systems understand the movement of objects in a video.

In autonomous driving, it helps detect moving vehicles and pedestrians. In video surveillance, it helps track people and detect unusual movements. In action recognition, it helps identify actions such as walking, running, and jumping.

Conclusion: Optical Flow is an important computer vision technique used to understand motion in video sequences. Lucas-Kanade is useful for fast feature tracking, while Farneback is useful for detailed motion analysis across the image.